Food and body-image themes in annotated social media posts

INFO 2950 · Phase II · Parts 1 and 2

Group: Anya Choudhary; Zara Islam (zsi2); Riya Gupta (rg823); April Vorontsova (vv252).

Github repo: AnyaChoudhary/info2950-fall2026-project

Key terms

• ED means eating disorder. Pro-ED describes content that presents disordered eating positively or as a goal.

• An annotation or code is a label assigned to content. It describes the post, not a diagnosis of its creator.

• Eligible posts are the posts a coding question applies to. Twitter restraint uses the 326 eating-concern posts as its denominator.

• A structural blank means the codebook skipped that question. It is not treated as a recorded zero.

• EDE means Eating Disorder Examination. EDTT uses its four category names: Restraint, Eating Concern, Shape Concern, and Weight Concern. These are content labels here, not assessments of the people posting.

• Shape concern refers to body shape or appearance. Eating concern refers to food and eating. Weight concern refers to body weight. These labels can overlap. A table's shape instead means rows and columns.

• Restraint refers to deliberately limiting food intake.

• Audio valence estimates musical positivity or brightness. Audio arousal estimates musical energy. Both range from 0 to 1 and do not measure a person's emotions.

• Harmonic ratio describes the proportion of harmonic sound in the audio. Tempo is measured in beats per minute.

• Format means video or photos (a photo slideshow) in EDTT.

1. Research questions

Our main question is: What food and body-image concerns appear in these posts, and how do they relate to other features of the posts?

1. Which concerns and content features appear in each dataset, and how common are they among posts where each label applies?

2. Among Twitter posts about eating disorders or body image, are shape-concern and eating-concern labels associated?

3. In EDTT, is a restraint label associated with audio valence after accounting for whether the post is a video or photo slideshow?

The exploratory analysis addressed question 1. Questions 2 and 3 are planned for the final report and have not been tested. The files describe posts, so they cannot show algorithm effects, engagement effects, diagnoses, or changes in a person's relationship with food. Recovery and social comparison are not measured consistently enough to use as shared outcomes.

Twitter provides a larger sample of coded posts; EDTT provides a smaller sample with visual and audio features. We analyzed them separately because their collection methods and labels differ.

Questions and data

Question	Dataset	What we measure	Possible predictors	Posts included
1	Twitter and EDTT, separately	Counts and percentages for each label	None, descriptive summaries	Depends on the label's coding rules
2	Twitter	Eating concern (eating)	Shape concern (shape)	2,584 related posts
3	EDTT	Audio valence (audio_valence, 0 to 1)	Restraint (ede_restraint) and post format (type)	79 posts with nonblank EDE text

All measures come from released fields or labels extracted directly from them. For question 3, the planned model will code type as photos versus video. A separate photos column has not been added to the cleaned data. None of these questions requires viewer, engagement, or exposure data.

2. Data collection and cleaning

2.1 Data sources

The source files were downloaded on October 15, 2026. We kept the original files in data/raw/ and recorded their source links and checksums so the analysis can be reproduced. The same unchanged files are hosted in the project GitHub raw-data folder, pinned to the verified upload commit so reviewers can retrieve the exact inputs.

Twitter: Cavazos-Rehg et al. (2019), PLOS ONE article, S1 Dataset, and S1 Codebook. The study sampled 3,000 English-language posts from a keyword-selected January 2015 corpus. The release supplies de-identified codes. The original post text is not included. The published methods describe trained coders and adjudication.

TikTok: Shaveet, Viranda, and Choudhury (2026), EDTT v2, public data, and data dictionary. This is a curated, expert-annotated pro-ED content sample assembled through text retrieval and recommendation surfacing. Private post identifiers and transcripts are withheld. The record was published in April 2026; the proposed 2024–2025 collection window was not verified in the public documentation inspected and is not assumed here.

Project files, cleaning outputs, and documentation are available in the GitHub repository.

No scraping, individual contact, account identification, or new annotation was performed. No API credentials were required.

Libraries

We used pandas to organize the data and matplotlib to create the charts. Below, we imported these libraries and the tools needed to locate and check the saved files.

Code cell [1]

```
# We imported the libraries used below.
from pathlib import Path
import hashlib, json, re, sys, platform
import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
from IPython.display import display, Markdown

from urllib.request import urlopen
import ssl
import certifi

from matplotlib import font_manager

# We displayed complete definitions in notebook tables.
```

```
pd.set_option('display.max_colwidth', None)
```

Data files

Below, we added a check for the saved data folder. The notebook can retrieve the same files from GitHub if they are missing, allowing it to run in Colab.

Code cell [2]

```
# We located the project folder and restore missing inputs.
tls_context = ssl.create_default_context(cafile=certifi.where())
candidates = [Path.cwd(), Path.cwd() / 'info2950_project', *Path.cwd().parents]
ROOT = next((p for p in candidates if (p / 'data/raw/manifest.json').exists()), None)
if ROOT is None:
    ROOT = Path.cwd() / 'info2950_project'
    raw_folder = ROOT / 'data/raw'
    raw_folder.mkdir(parents=True, exist_ok=True)
    commit = '905e574351e821300dc5b199747216a3bd630e1b'
    base = ('https://raw.githubusercontent.com/AnyaChoudhary/'
            f'info2950-fall2026-project/{commit}/data/raw/')
    try:

        with urlopen(base + 'manifest.json', timeout=45, context=tls_context) as response:
            manifest_bytes = response.read()
        manifest_hash = '05debbe74cce74f95992e9eccc3c333843031b3bcc65f50ccae8292e04be163c'
        assert hashlib.sha256(manifest_bytes).hexdigest() == manifest_hash
        saved_inputs = json.loads(manifest_bytes)
        for item in saved_inputs:
            name = item['file']
            assert Path(name).name == name, 'Unexpected input filename.'
            with urlopen(base + name, timeout=45, context=tls_context) as response:
                contents = response.read()
            assert hashlib.sha256(contents).hexdigest() == item['sha256'], name
            (raw_folder / name).write_bytes(contents)

        # We saved the manifest last so an interrupted restore can be retried.
        (raw_folder / 'manifest.json').write_bytes(manifest_bytes)
    except Exception as exc:
        raise RuntimeError(
            'Could not restore the saved inputs. Check the internet connection '
            'and rerun, or extract the full project bundle before running.'
        ) from exc
    print('Restored the four verified source files from the saved GitHub snapshot.')
RAW = ROOT / 'data/raw'
INTER = ROOT / 'data/intermediate'
CLEAN = ROOT / 'data/processed'
FIG = ROOT / 'figures'

for folder in [INTER, CLEAN, FIG]:
    folder.mkdir(parents=True, exist_ok=True)
```

File checks

Below, we checked the source files against their saved checksums to confirm that the inputs had not changed.

Code cell [3]

```
# We verified all raw files.
manifest = json.loads((RAW / 'manifest.json').read_text())
for item in manifest:
    actual = hashlib.sha256((RAW / item['file']).read_bytes()).hexdigest()
    assert actual == item['sha256'], item['file']
assert hashlib.md5((RAW / 'EDTT_public.csv').read_bytes()).hexdigest() ==
'8bf6f87fd0ff4147790209f66ee364ea'

print('All four source files passed the checksum checks.')
```

```
All four source files passed the checksum checks.
```

Charts

We used charts to compare the post labels and audio values. Below, we set their appearance and added a function to save each figure.

Code cell [4]

```
# Prefer Times New Roman; keep a portable serif fallback for other computers.
font_folder = Path('/System/Library/Fonts/Supplemental')
for font_file in font_folder.glob('Times New Roman*.ttf'):
    font_manager.fontManager.addfont(str(font_file))
plt.rcParams.update({'font.family': 'serif',
                     'font.serif': ['Times New Roman', 'Times', 'DejaVu Serif'],
                     'text.color': 'black', 'axes.labelcolor': 'black',
                     'axes.titlecolor': 'black', 'xtick.color': 'black',
                     'ytick.color': 'black', 'figure.dpi': 130, 'savefig.dpi': 180,
                     'font.size': 10, 'axes.spines.top': False,
                     'axes.spines.right': False, 'axes.titleweight': 'bold'})
BLUE, TEAL, ORANGE = '#315B86', '#16847D', '#C87A36'

def finish(name):
    plt.savefig(FIG / (name + '.png'), bbox_inches='tight')
    plt.show()
print(f'Python {platform.python_version()} | pandas {pd.__version__} | '
      f'numpy {np.__version__} | matplotlib {matplotlib.__version__}')
```

```
Python 3.12.0 | pandas 3.0.6 | numpy 2.5.4 | matplotlib 3.11.2
```

2.2 Twitter cleaning

shape is a content label for concern about body shape or appearance, such as a focus on thinness or specific body areas. It is different from a table's .shape, which reports its number of rows and columns. The content label does not diagnose a person.

We kept all original columns so each result can be traced to the source. The cleaned analysis file includes only posts coded as related to eating disorders or body image (ed_about = 1). We kept all 3,000 source rows in the intermediate file.

Reading the Twitter data

Below, we read the data, standardized the column names, and checked the IDs and recorded values. We preserved blank entries. Most labels use 0 or 1, while ed_about also uses 9 for unknown relevance.

Code cell [5]

```
# We checked that the labels follow the source definitions.
tw_raw = pd.read_csv(RAW / 'twitter_s1.csv')
tw = tw_raw.copy()
tw.columns = tw.columns.str.strip().str.lower()
assert tw['id'].notna().all() and tw['id'].is_unique
for col in tw:
    tw[col] = pd.to_numeric(tw[col], errors='raise').astype('Int64')
    if col != 'id':
        allowed = {0, 1, 9} if col == 'ed_about' else {0, 1}
        assert set(tw[col].dropna()).issubset(allowed), col
```

Check when each label applies

Image labels apply to shape-concern posts. Body-image details apply only when those posts have an image. Restraint and binge/purge labels apply only to eating-concern posts. Aversion follows the source's noconcern rule.

Below, we checked that labels were present where required and blank where the codebook skipped them. We kept these skipped entries blank rather than changing them to zeros.

Code cell [6]

```
# We checked the codebook skip rules.
eligible = {'ed_about': pd.Series(True, index=tw.index)}
for col in ['shape', 'eating', 'weight', 'noconcern']:
    eligible[col] = (tw['ed_about'] == 1)
eligible['noimage'] = (tw['shape'] == 1).fillna(False)
for col in ['body', 'thighs', 'stomach', 'hipbones', 'ribs', 'collarbones', 'extreme']:
    eligible[col] = ((tw['shape'] == 1) & (tw['noimage'] == 0)).fillna(False)
for col in ['restraint', 'purge']:
    eligible[col] = (tw['eating'] == 1).fillna(False)
eligible['aversion'] = (tw['noconcern'] == 1).fillna(False)
audit = []
for col, mask in eligible.items():

    inside_missing = int(tw.loc[mask, col].isna().sum())
    outside_recorded = int(tw.loc[~mask, col].notna().sum())
    assert inside_missing == 0 and outside_recorded == 0, col
    audit.append([col, int(mask.sum()), int((~mask).sum()), inside_missing])
twitter_audit = pd.DataFrame(audit, columns=[
    'field', 'eligible n', 'outside universe n', 'unexpected missing n'])
display(twitter_audit)
```

	field	eligible n	outside universe n	unexpected missing n
0	ed_about	3000	0	0
1	shape	2584	416	0
2	eating	2584	416	0
3	weight	2584	416	0
4	noconcern	2584	416	0
5	noimage	1679	1321	0
6	body	861	2139	0
7	thighs	861	2139	0
8	stomach	861	2139	0
9	hipbones	861	2139	0
10	ribs	861	2139	0
11	collarbones	861	2139	0
12	extreme	861	2139	0
13	restraint	326	2674	0
14	purge	326	2674	0
15	aversion	599	2401	0

Keep related posts and flag conflicts

The analysis includes posts coded as related to eating disorders or body image (ed_about = 1). Eleven source rows say noconcern = 0 even though shape, eating, and weight are all zero. We flagged them and preserved their original labels because the released data cannot tell us which label is wrong. Repeated code patterns are not removed: different posts can share the same labels.

Code cell [7]

```
# We saved the full validated table and the related-post subset.
tw.to_csv(INTER / 'twitter_validated.csv', index=False)
twitter = tw.loc[(tw['ed_about'] == 1)].copy()
# We flagged conflicting labels without changing the originals.
expected_none = (twitter[['shape', 'eating', 'weight']] == 0).all(axis=1).astype(int)
logic_mismatch = (twitter['noconcern'] != expected_none)
twitter['noconcern_inconsistent'] = logic_mismatch.astype('Int64')
twitter.loc[logic_mismatch, ['id', 'noconcern_inconsistent']].to_csv(
    CLEAN / 'twitter_logic_flags.csv', index=False)
print('Rows with inconsistent noconcern code:', int(logic_mismatch.sum()))
twitter.to_csv(CLEAN / 'twitter_related.csv', index=False)
display(tw['ed_about'].value_counts().rename_axis('ed_about').to_frame('n'))

print('Unique IDs:', tw.id.nunique(), '| Full-row duplicates:', tw.duplicated().sum())
print('Retained related posts:', len(twitter), '| Excluded unrelated/unknown:', len(tw)-len(twitter))
```

Rows with inconsistent noconcern code: 11

	n
ed_about	
1	2584
0	342
9	74

Unique IDs: 3000 | Full-row duplicates: 0
Retained related posts: 2584 | Excluded unrelated/unknown: 416

2.3 TikTok cleaning

The public EDTT data do not include post IDs. We assigned record_id from the original row order solely to track this file. We checked exact duplicates but did not infer duplicate posts from similar summaries or shared audio.

We standardized column names, trimmed surrounding whitespace, preserved annotation wording, checked numeric ranges and post format, and derived indicators from explicit category headers. Header matching is case-insensitive and allows a header after either a semicolon or a colon. This accommodates an observed separator inconsistency in the visual annotations. We left item-level spelling unchanged and did not reinterpret the annotation text.

A blank EDE subscale field remains unknown for all four derived EDE indicators. It is not recoded to four zeros. A zero on a nonblank field means the category was not recorded there, not that a clinical symptom is absent. We also preserved blank diagnostic fields because the dictionary did not explain their meaning. We checked the audio labels against the dictionary's thresholds and reported disagreements without changing the original values.

Read and tidy the columns

Below, we retained all 13 source fields, removed extra spaces, and standardized the column names. An added row number identifies each record within this file. It is not a TikTok post ID.

Code cell [8]

```
# We tidied column names and keep the original text and blank entries.
tt_raw = pd.read_csv(RAW / 'EDTT_public.csv')
tt = tt_raw.copy()
tt.columns = [re.sub(r'[^a-z0-9]+', '_', c.lower()).strip('_') for c in tt.columns]
for col in tt.select_dtypes(include=['object', 'string']):
    tt[col] = tt[col].astype('string').str.strip().replace('', pd.NA)
tt.insert(0, 'record_id', range(1, len(tt)+1))
```

Check format and audio values

Below, we checked the post formats and audio values against the source definitions. Tempo must be positive, and scores defined on a 0-to-1 scale must fall within that range. We saved this version before adding category labels.

Code cell [9]

```
# We checked the source fields.
assert tt['type'].isin(['video', 'photos']).all()
nums = ['audio_tempo_bpm', 'audio_valence', 'audio_arousal', 'harmonic_ratio', 'mode_major']
for col in nums:
    tt[col] = pd.to_numeric(tt[col], errors='raise')
assert tt[nums].notna().all().all()
assert (tt['audio_tempo_bpm'] > 0).all()
for col in ['audio_valence', 'audio_arousal', 'harmonic_ratio']:
    assert tt[col].between(0, 1).all(), col
assert tt['mode_major'].isin([0, 1]).all()
tt.to_csv(INTER / 'edtt_normalized.csv', index=False)
```

Turn category headers into labels

Below, we extracted four EDE labels and seven visual labels from the headings in the original annotations. We coded a recorded header as 1 and an absent header in nonblank text as 0. We kept blank EDE text unknown in all four labels. Header matching handles the separator variations found in the source without rewriting the annotation text.

Code cell [10]

```
# We extracted only the named category headers.
def header_flag(series, header):
    pattern = r'(?:^|[;:])\s*' + re.escape(header) + r'\s*:'
    return series.str.contains(pattern, case=False, regex=True).astype('Int64')

ede_headers = {'shape': 'Shape Concern', 'eating': 'Eating Concern',
               'weight': 'Weight Concern', 'restraint': 'Restraint'}
visual_headers = {'body_checking': 'Body Checking & Self Emphasis',
 'on_screen_text': 'On-Screen Text', 'performance': 'Performance & Body Language',
 'weight_imagery': 'Weight-Related Imagery', 'food_imagery': 'Food-Related Imagery',
 'media_memes': 'Media & Memes', 'summaries_metrics': 'Summaries & Metrics'}
for key, header in ede_headers.items():

    tt['ede_' + key] = header_flag(tt['ede_subscale_items'], header)
for key, header in visual_headers.items():
    tt['visual_' + key] = header_flag(tt['visual_elements'], header)
ede_cols = ['ede_' + k for k in ede_headers]
visual_cols = ['visual_' + k for k in visual_headers]
# We confirmed all nonblank annotations contain recognized categories.
assert tt.loc[tt.ede_subscale_items.notna(), ede_cols].sum(axis=1).gt(0).all()
assert tt[visual_cols].sum(axis=1).gt(0).all()
assert tt.loc[tt.ede_subscale_items.isna(), ede_cols].isna().all().all()
tt['ede_category_count'] = tt[ede_cols].sum(axis=1, min_count=4).astype('Int64')
```

Check audio labels and save

Below, we compared the low, medium, and high audio labels with the published thresholds. We reported any disagreement without changing the source label and kept all 85 posts in the saved data.

Code cell [11]

```
# We checked audio categories and save the cleaned table.
label_audit = []
for col in ['audio_valence', 'audio_arousal']:
    expected_labels = []
    for value in tt[col]:
        if value < .33:
            expected_labels.append('low')
        elif value <= .66:
            expected_labels.append('medium')
        else:
            expected_labels.append('high')
    expected = pd.Series(expected_labels, index=tt.index)

    label_audit.append([col, int((expected != tt[col + '_label']).sum())])
tiktok = tt.copy()
tiktok.to_csv(CLEAN / 'edtt_features.csv', index=False)
display(pd.DataFrame(label_audit, columns=['audio field', 'label disagreements']))
display(tt_raw.isna().sum().rename_axis('raw field').to_frame('blank n'))
print('EDTT rows retained:', len(tiktok), '| Exact raw duplicates:', tt_raw.duplicated().sum())
print('Known EDE annotation:', tt.ede_subscale_items.notna().sum(),
      '| Blank EDE annotation:', tt.ede_subscale_items.isna().sum())
```

	audio field	label disagreements
0	audio_valence	0
1	audio_arousal	0

	blank n
raw field	
Type	0
Summary	0
EDE Subscale Items	6
EDE Diagnostic Items	58
Visual Elements	0
Audio Elements	0
Audio Tempo (bpm)	0
Audio Valence	0
Audio Valence Label	0
Audio Arousal	0
Audio Arousal Label	0
Harmonic Ratio	0
Mode Major	0

```
EDTT rows retained: 85 | Exact raw duplicates: 0
Known EDE annotation: 79 | Blank EDE annotation: 6
```

2.4 What we kept

The cleaning produced two separate analysis tables. We kept the datasets separate because their posts cannot be matched and their collection methods differ. Eleven Twitter rows have noconcern=0 while all three concern fields are zero. We flagged these rows in noconcern_inconsistent, retained the original codes, and saved their synthetic IDs separately. The aversion denominator continues to follow the original source eligibility code, not a reconstructed universe. Without the original posts, we cannot determine which label is wrong. No rows were removed from EDTT; Twitter excluded only the 342 unrelated and 74 unknown-relevance rows from its main analysis. We did not fill missing values, remove outliers, remove repeated annotation patterns, or change structural blanks to zero.

Intermediate files preserve the complete validated Twitter release and normalized EDTT release. Processed files preserve the variables needed for future analysis. The table below records the number of posts retained. Any later changes to these exclusions will be documented.

Why we kept these columns

We kept every source column. Keeping a column for traceability does not mean using it in a model.

Columns	Why we kept them	Their role in the analysis
Twitter id, ed_about	Track rows and explain inclusion	ID is not a predictor, ed_about selects related posts
Twitter shape, eating, weight	Describe the main concern labels	EDA, shape and eating support question 2
Twitter image, restraint, purge, and aversion codes	Preserve the remaining source labels	Descriptive summaries use each label's eligible posts
Twitter noconcern and added conflict flag	Show source coding conflicts	Audit the data without changing original codes
EDTT type and audio fields	Describe format and sound	EDA, format and valence support question 3
EDTT text and annotation fields	Trace derived labels to their source	Parse category headers, do not quote sensitive summaries
EDTT diagnostic text	Preserve the original release	Not used in analysis because blank values are unclear
EDTT row number, category labels, and count	Track rows and summarize recorded categories	EDA, ede_restraint supports question 3

Twitter excluded 342 unrelated and 74 unknown-relevance posts from the analysis table. EDTT excluded no rows. We did not fill in missing values, remove outliers, or combine the sources.

Code cell [12]

```
# We recorded how many posts remain and why any were excluded.
flow = pd.DataFrame([
 ['Twitter', len(tw_raw), len(twitter), len(tw_raw)-len(twitter), 'ed_about must equal 1'],
 ['EDTT', len(tt_raw), len(tiktok), 0, 'retain all public rows']
], columns=['source', 'raw n', 'analysis n', 'excluded n', 'rule'])
flow.to_csv(CLEAN / 'row_flow.csv', index=False)
twitter_audit.to_csv(CLEAN / 'twitter_eligibility_audit.csv', index=False)
display(flow)
print(f'Twitter: {twitter.shape[0]} rows, {twitter.shape[1]} columns')
print(f'EDTT: {tiktok.shape[0]} rows, {tiktok.shape[1]} columns')
```

	source	raw n	analysis n	excluded n	rule
0	Twitter	3000	2584	416	ed_about must equal 1
1	EDTT	85	85	0	retain all public rows

Twitter: 2584 rows, 18 columns
EDTT: 85 rows, 26 columns

Final data checks

Below, we checked the number of rows and columns and confirmed that the missing-label rules were preserved. We used these two tables throughout the exploratory analysis.

Code cell [13]

```
# We checked the final tables before exploring the data.
assert twitter.shape == (2584, 18)
assert tiktok.shape == (85, 26)
assert (twitter['ed_about'] == 1).all()
assert twitter['id'].is_unique
assert int(twitter['noconcern_inconsistent'].sum()) == 11
assert int(tiktok['ede_subscale_items'].isna().sum()) == 6
assert tiktok.loc[tiktok['ede_subscale_items'].isna(), ede_cols].isna().all().all()
assert list(tw_raw.columns.str.strip().str.lower()) == list(twitter.columns[:17])
print('Both cleaned tables are ready for EDA.')
```

Both cleaned tables are ready for EDA.

3. Data description

What a row means

Each row is one post, not one person or account. Percentages and planned models describe posts in the samples. Multiple posts could come from the same person, but the released files do not identify those groups.

Rows and columns

Table	Rows	Columns	What is included
Twitter raw	3,000	17	Row ID and 16 annotation codes
Twitter cleaned	2,584	18	All 17 source columns plus a conflict flag, only related posts
EDTT raw	85	13	Format, five text fields, five numeric audio fields, and two audio labels
EDTT cleaned	85	26	All 13 source fields, row number, four EDE labels, seven visual labels, and an EDE category count

Twitter has more posts but fewer kinds of measures. Its ID is an identifier. Its relevance code allows 0, 1, or 9. The remaining codes are 0 or 1 where applicable. EDTT has fewer posts and more text and audio features. Only 79 posts have nonblank EDE text, which limits the precision of the planned model.

Why the datasets were made

Twitter: One row represents one sampled post. The analysis table has 2,584 rows and retains the 17 source columns plus a logical-inconsistency flag. id is a synthetic row identifier. The remaining fields are categorical annotations, not symptom severity scores. The release was created to characterize ED/body-image expression. Keyword selection and conditional coding determine which posts and features are represented. Multiple posts may come from one person, but this release cannot identify clusters.

EDTT: One row represents one curated TikTok post (video or photo slideshow). The release was created to study multimodal pro-ED content. The processed table has 85 rows. It keeps all 13 source fields and adds a row number, 11 category labels, and a count of recorded EDE categories. Text fields contain summaries and annotations, not the original private transcript. Audio valence and arousal are heuristic musical features, not the emotions of a creator or viewer. The original five numeric fields are tempo in beats per minute, valence, arousal, harmonic ratio, and major/minor mode.

No common clinical scale: Twitter's restraint field is assessed only inside eating-concern posts, whereas EDTT lists restraint as a separate category. Even shared labels do not establish identical measurement. We retained each source's definitions.

Privacy and consent

The Twitter paper reports public posts and an institutional determination that its study did not involve human subjects. That does not show that every poster knew about this specific secondary analysis or consented to it. The EDTT release describes privacy protections and withheld private fields. The inspected documentation does not establish individual notification or consent. We used aggregate summaries, avoided quoting sensitive post summaries, and made no attempts to identify creators. Post annotations cannot diagnose the people who posted them.

What the data can show

These tables support descriptive secondary analysis and prespecified associations between annotations within each sample. They are unsuitable for diagnosing creators, evaluating moderation decisions about particular accounts, estimating engagement, or ranking platforms by harm. There is no predefined train/test split. No prediction benchmark or model training is performed in Phase II. The analysis-ready files preserve source annotations so a reviewer can trace every derived flag back to its input. Questions about the original annotation or collection should be directed to the source authors rather than inferred from the summaries.

Cleaning and missing values

Twitter column names were lowercased and codes converted to nullable integers. We checked allowed values and skip rules, kept 2,584 related posts, and flagged 11 conflicts without changing them. There are no unexpected blanks under the codebook rules. For example, restraint is blank for 2,674 of the original 3,000 rows because it is coded only for the 326 posts with a recorded eating concern. The other rows include unrelated and unknown-relevance posts, not just posts coded as having no eating concern.

For EDTT, we standardized names, trimmed spaces, checked ranges, and extracted four EDE and seven visual category labels. The audio labels match the published thresholds, with zero disagreements. Six blank EDE subscale fields remain unknown. Another 58 rows have blank EDE diagnostic text. Its meaning is not explained, so diagnostic text is retained but not analyzed.

Neither dataset was merged, imputed, or filtered for outliers. Similar annotation patterns were retained. The cleaning steps and saved intermediate files are in Sections 2.2 through 2.4.

Column guide

The tables below describe the source fields and added columns. Blank values follow the rules described in Section 2.

Code cell [14]

```
# We described what each Twitter column means.
tw_dictionary = [
 ('id', 'Synthetic unique row ID', 'All 3,000'),
 ('ed_about', '0 unrelated; 1 related; 9 unknown', 'All 3,000'),
 ('shape / eating / weight', 'Concern recorded: 0 no, 1 yes', 'Related posts'),
 ('noconcern', 'None of shape/eating/weight: 1 yes', 'Related posts'),
 ('noimage', '0 image; 1 no image', 'Shape concern'),
 ('body / thighs / stomach', 'Depicted region: 0 no, 1 yes', 'Shape + image'),
 ('hipbones / ribs / collarbones', 'Depicted region: 0 no, 1 yes', 'Shape + image'),
 ('extreme', 'Extreme thinness annotation: 0/1', 'Shape + image'),
 ('restraint / purge', 'Restraint or binge/purge annotation: 0/1', 'Eating concern'),
 ('aversion', 'Aversion to pro-ED content: 0/1', 'Source noconcern=1'),

 ('noconcern_inconsistent', 'Added flag for disagreement with three concern codes', 'Related posts')]
display(pd.DataFrame(tw_dictionary, columns=['Twitter field', 'meaning', 'applies to']))
```

	Twitter field	meaning	applies to
0	id	Synthetic unique row ID	All 3,000
1	ed_about	0 unrelated; 1 related; 9 unknown	All 3,000
2	shape / eating / weight	Concern recorded: 0 no, 1 yes	Related posts
3	noconcern	None of shape/eating/weight: 1 yes	Related posts
4	noimage	0 image; 1 no image	Shape concern
5	body / thighs / stomach	Depicted region: 0 no, 1 yes	Shape + image
6	hipbones / ribs / collarbones	Depicted region: 0 no, 1 yes	Shape + image
7	extreme	Extreme thinness annotation: 0/1	Shape + image
8	restraint / purge	Restraint or binge/purge annotation: 0/1	Eating concern
9	aversion	Aversion to pro-ED content: 0/1	Source noconcern=1
10	noconcern_inconsistent	Added flag for disagreement with three concern codes	Related posts

TikTok columns

We retained the original annotation text alongside the added category labels so the extraction can be checked.

Code cell [15]

```
# We described the original and added TikTok columns.
tt_dictionary = [
 ('type', 'video or photos'), ('summary', 'Public descriptive summary'),
 ('ede_subscale_items', 'Recorded EDE category/item text'),
 ('ede_diagnostic_items', 'Recorded diagnostic-feature text; not diagnoses'),
 ('visual_elements / audio_elements', 'Visual annotations / audio description'),
 ('audio_tempo_bpm', 'Audio tempo, beats per minute'),
 ('audio_valence / audio_arousal', 'Heuristic musical positivity / energy, 0–1'),
 ('audio_valence_label / audio_arousal_label', 'low <.33; medium .33–.66; high >.66'),
 ('harmonic_ratio', 'Harmonic sound proportion, 0–1'),
 ('mode_major', '1 major; 0 minor'),
 ('record_id', 'Added sequential row number; not a platform ID'),

 ('ede_shape / ede_eating / ede_weight / ede_restraint', 'Header present=1; absent=0; blank source=NA'),
 ('visual_* (seven indicators)', 'Header present=1; absent=0'),
 ('ede_category_count', 'Number of four headers, 0–4; blank source=NA')]
display(pd.DataFrame(tt_dictionary, columns=['EDTT field', 'meaning']))
```

	EDTT field	meaning
0	type	video or photos
1	summary	Public descriptive summary
2	ede_subscale_items	Recorded EDE category/item text
3	ede_diagnostic_items	Recorded diagnostic-feature text; not diagnoses
4	visual_elements / audio_elements	Visual annotations / audio description
5	audio_tempo_bpm	Audio tempo, beats per minute
6	audio_valence / audio_arousal	Heuristic musical positivity / energy, 0–1
7	audio_valence_label / audio_arousal_label	low <.33; medium .33–.66; high >.66
8	harmonic_ratio	Harmonic sound proportion, 0–1
9	mode_major	1 major; 0 minor
10	record_id	Added sequential row number; not a platform ID
11	ede_shape / ede_eating / ede_weight / ede_restraint	Header present=1; absent=0; blank source=NA
12	visual_* (seven indicators)	Header present=1; absent=0
13	ede_category_count	Number of four headers, 0–4; blank source=NA

4. Limitations

• Selected samples: Both datasets focused on eating-disorder-related content. Their frequencies do not represent all posts or social media users.

• Different sources: Collection methods, formats, dates, and coding rules differed. We could not interpret differences between the samples as platform effects or changes over time.

• Missing outcomes: Neither dataset included engagement, viewer exposure, or changes in food attitudes. These outcomes could not be studied with the available data.

• Imperfect labels: Annotations reflect how researchers interpreted each post. The 11 conflicting Twitter records could not be resolved without the original content.

• Missing annotations: Twitter blanks followed the coding rules, while six EDTT category fields had an unclear meaning. Treating these blanks as zero would change the denominators and misrepresent the evidence.

• Small sample: Only 79 EDTT posts had the annotations needed for the planned model. Estimates from this group may be imprecise.

• Repeated creators: Posts may share creators or audio. Without reliable identifiers, we could not account for this dependence, which may make confidence intervals too narrow.

• Posts are not diagnoses: Content labels and musical scores do not establish a person's symptoms or feelings. Treating them as diagnoses could stigmatize creators.

• Privacy: We reported aggregate results and avoided reproducing sensitive post summaries. Public availability did not establish that creators expected this use of their posts.

• Removed content: Posts removed before collection may be absent. The files do not include removal histories, so the extent of this bias remains unknown.

• Language and time: Twitter included English-language posts from January 2015. Changes in language and platform practices limit how well the sample describes current content.

• Different definitions: The research teams used separate codebooks. Similar category names do not guarantee equivalent measurements across the datasets.

We therefore focused on patterns within each sample. The planned models will describe associations between post features, not effects on people's health or attitudes.

5. Exploratory data analysis

5.1 Twitter concerns

We first examined how often each concern appears. Since a post can have several labels, the percentages need not add to 100. We counted shape, eating, and weight separately. Their co-occurrence is reserved for the planned Twitter analysis.

Count the labels

For each label, we divided the number of recorded cases by the number of posts where that label applies. We excluded blank entries from each denominator without treating them as zeros.

Code cell [16]

```
# We counted posts with each label, using only posts where it applies.
def count_labels(posts, labels):
    results = []
    for column, label in labels.items():
        posts_counted = posts[column].notna().sum()
        posts_with_label = (posts[column] == 1).sum()
        percent = 100 * posts_with_label / posts_counted
        results.append([
            label, posts_with_label, posts_counted, percent
        ])

    return pd.DataFrame(results, columns=[
        'Label', 'Count', 'Eligible posts', "Percent"
    ])
```

Choose the groups

The main labels use all 2,584 related posts. Restraint and binge/purge use eating-concern posts. Extreme thinness uses shape-concern posts with images. Aversion uses the source's no-concern group. Aversion means opposition to pro-ED content, not necessarily recovery.

Code cell [17]

```
# We counted the main labels and the labels with narrower eligibility.
main_concerns = count_labels(twitter, {
    'shape': 'Shape concern',
    'eating': 'Eating concern',
    'weight': 'Weight concern'
})

other_concerns = count_labels(twitter, {
    'restraint': 'Restraint | eating',
    'purge': 'Binge/purge | eating',
    'extreme': 'Extreme thinness | shape + image',
    'aversion': 'Aversion | no concerns'

})
```

Plot the percentages

Horizontal bars keep labels readable. Both panels use the same percentage scale, but their denominators differ. The table gives the counts behind each bar.

Code cell [18]

```
# Same percentage scale for both panels.
figure, charts = plt.subplots(
    1, 2, figsize=(15,5), layout='constrained'
)

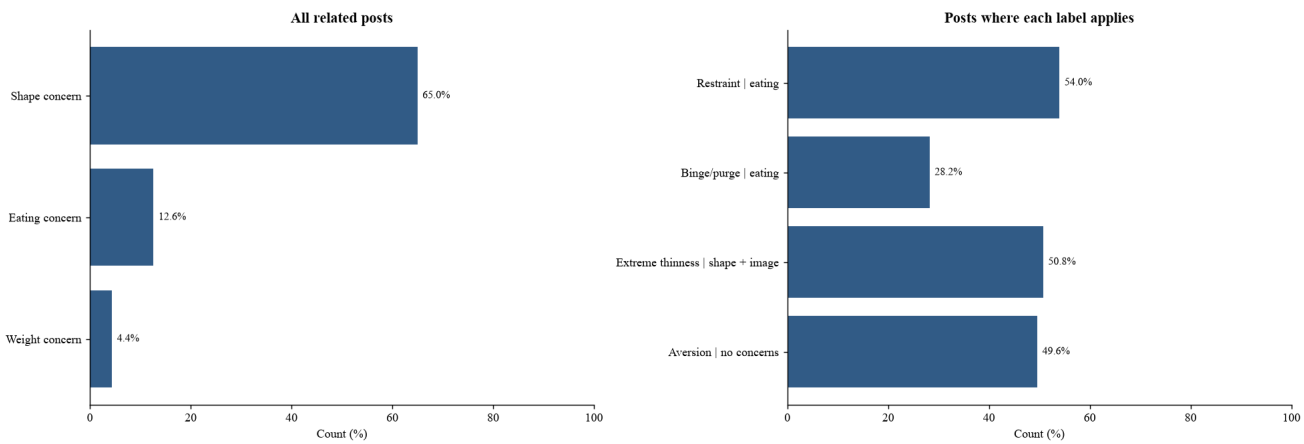
for chart, table, title in zip(
    charts,
    [main_concerns, other_concerns],
    ['All related posts', 'Posts where each label applies']
):
    chart.barh(table['Label'], table["Percent"], color=BLUE)
    chart.invert_yaxis()

    chart.set_xlim(0, 100)
    chart.set_xlabel('Count (%)')
    chart.set_title(title)

    for row, percent in enumerate(table["Percent"]):
        chart.text(
            percent + 1, row, f'{percent:.1f}%',
            ha='left', va='center', fontsize=9
        )

finish('01_twitter_themes')

display(pd.concat(
    [main_concerns, other_concerns], ignore_index=True
).round(1))
```



	Label	Count	Eligible posts	Percent
0	Shape concern	1679	2584	65.0
1	Eating concern	326	2584	12.6
2	Weight concern	113	2584	4.4
3	Restraint | eating	176	326	54.0
4	Binge/purge | eating	92	326	28.2

	Label	Count	Eligible posts	Percent
5	Extreme thinness | shape + image	437	861	50.8
6	Aversion | no concerns	297	599	49.6

Shape concern appears in 1,679 of 2,584 related posts (65.0%), compared with 326 eating-concern posts (12.6%) and 113 weight-concern posts (4.4%). Restraint appears in 176 of the 326 eating-concern posts (54.0%).

Body image remains central to question 1, while eating concerns form a smaller group for question 2. These are frequencies of post labels, not rates of illness among people.

5.2 TikTok categories and blanks

EDTT has 79 posts with EDE category text and six with blank fields. We left those six unknown. This section answers question 1 and explains why question 3 currently has 79 usable posts.

Count categories and missing labels

For each category, we calculated the percentage among known annotations and the possible range across all 85 posts. The minimum assumes none of the six unknown posts has that category. The maximum assumes all six do.

Code cell [19]

```
# We counted EDE labels without treating missing annotations as "no."
category_counts = count_labels(tiktok, {
    'ede_shape': 'Shape concern',
    'ede_eating': 'Eating concern',
    'ede_weight': 'Weight concern',
    'ede_restraint': 'Restraint'
})

missing_labels = tiktok['ede_subscale_items'].isna().sum()
total_posts=len(tiktok)

category_counts['All posts: minimum %'] = (
    100 * category_counts['Count'] / total_posts
)
category_counts['All posts: maximum %'] = (
    100 * (
        category_counts['Count'] + missing_labels
    ) / total_posts
)
```

Show what the blanks could change

Teal dots use the 79 known annotations. Blue diamonds show the recorded count divided by all 85 posts. Gray lines show the possible all-post range if the missing categories were known. These are missing-data scenarios, not confidence intervals.

Code cell [20]

```
# Gray lines show what could change if the missing labels were known.
figure, chart = plt.subplots(
    figsize=(10, 5), layout='constrained'
)

chart.hlines(
    category_counts["Label"],
    category_counts['All posts: minimum %'],
    category_counts['All posts: maximum %'],
    color='gray', linewidth=6,
    label='Possible range across all 85 posts'
)

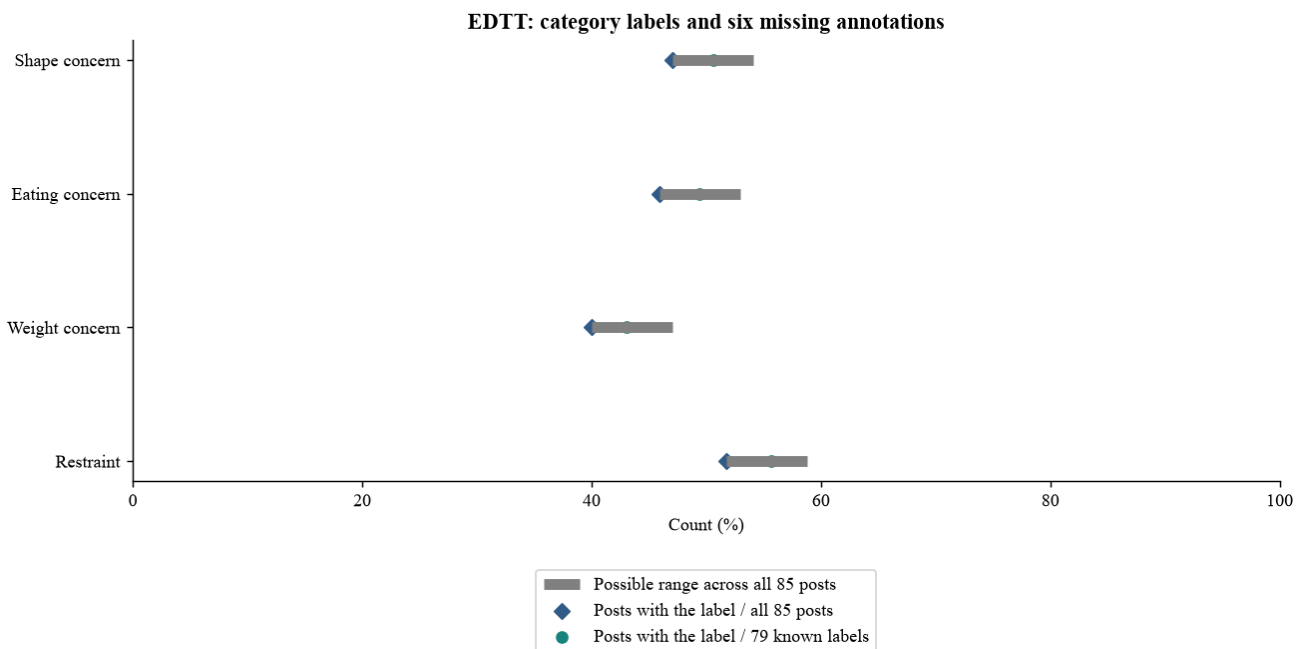
chart.scatter(
    category_counts['All posts: minimum %'],
    category_counts["Label"],
    color=BLUE, marker='D',
    label='Posts with the label / all 85 posts'
)

chart.scatter(
    category_counts['Percent'],
    category_counts["Label"],
    color=TEAL,

    label='Posts with the label / 79 known labels'
)

chart.invert_yaxis()
chart.set_xlim(0, 100)
chart.set_xlabel('Count (%)')
chart.set_title('EDTT: category labels and six missing annotations')
chart.legend(loc='upper center', bbox_to_anchor=(0.5, -0.18))

finish('02_edtt_ede')
display(category_counts.round(1))
```



	Label	Count	Eligible posts	Percent	All posts: minimum %	All posts: maximum %
0	Shape concern	40	79	50.6	47.1	54.1

	Label	Count	Eligible posts	Percent	All posts: minimum %	All posts: maximum %
1	Eating concern	39	79	49.4	45.9	52.9
2	Weight concern	34	79	43.0	40.0	47.1
3	Restraint	44	79	55.7	51.8	58.8

Restraint is the most common recorded category: 44 of 79 posts (55.7%). Shape concern appears in 40 (50.6%), eating concern in 39 (49.4%), and weight concern in 34 (43.0%). Categories can overlap.

Each all-post range spans 7.1 percentage points because six of 85 annotations are unknown. Treating those blanks as zero would hide this uncertainty. We have not compared any EDE category with audio valence.

5.3 Videos and photo posts

We then compared visual features in the 47 videos and 38 photo posts. Percentages are calculated within each group to account for their different sizes.

Count features by format

The table records counts, group sizes, percentages, and the percentage-point difference. The full table is saved for checking individual values.

Code cell [21]

```
# We compared counts and percentages for videos and photo posts.
video_posts = tiktok[tiktok["type"] == 'video']
photo_posts = tiktok[tiktok["type"] == 'photos']

visual_summary = pd.DataFrame(index=visual_cols)

visual_summary['Videos with this label'] = (
    video_posts[visual_cols].sum()
)
visual_summary['Videos counted'] = len(video_posts)
visual_summary['Videos (%)'] = (
    100 * visual_summary['Videos with this label'] / len(video_posts)

)

visual_summary['Photo posts with this label'] = (
    photo_posts[visual_cols].sum()
)
visual_summary['Photo posts counted'] = len(photo_posts)
visual_summary['Photo posts (%)'] = (
    100 * visual_summary['Photo posts with this label'] / len(photo_posts)
)

visual_summary['Photo % minus video %'] = (
    visual_summary['Photo posts (%)'] - visual_summary['Videos (%)']

)

visual_summary.index = list(visual_headers.values())
visual_summary.index.name = 'Visual feature'


visual_summary.to_csv(CLEAN / 'visual_format_summary.csv')
display(visual_summary[['Videos with this label', 'Photo posts with this label', 'Videos (%)', 'Photo
posts (%)']].round(1))
```

	Videos with this label	Photo posts with this label	Videos (%)	Photo posts (%)
Visual feature				
Body Checking & Self Emphasis	24	4	51.1	10.5
On-Screen Text	43	38	91.5	100.0
Performance & Body Language	37	11	78.7	28.9
Weight-Related Imagery	16	4	34.0	10.5
Food-Related Imagery	5	16	10.6	42.1
Media & Memes	10	27	21.3	71.1
Summaries & Metrics	1	14	2.1	36.8

Compare the formats

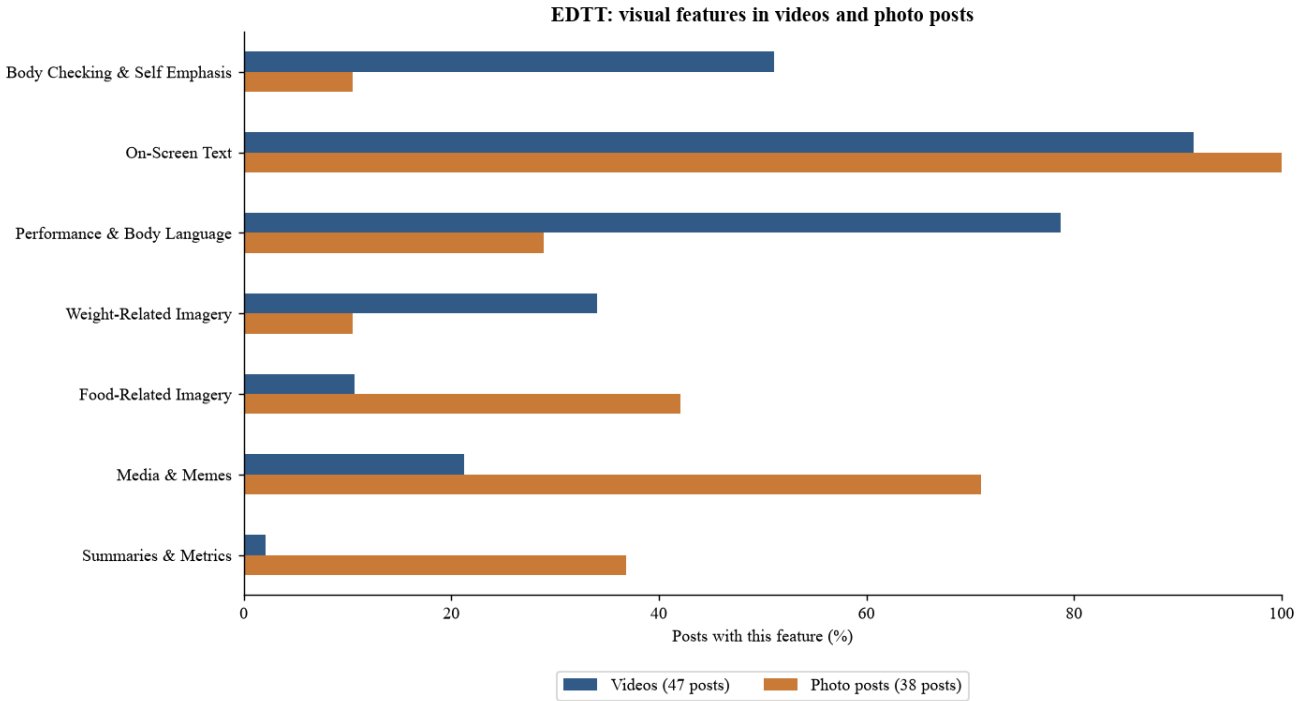
Each pair of bars shows one visual feature. The labels follow the source categories. This explores question 1 and supports keeping format in the planned audio model, but does not establish that format explains a restraint-valence association.

Code cell [22]

```
# We plotted paired bars directly from the percentage columns.
chart = visual_summary[['Videos (%)', 'Photo posts (%)']].plot.barh(
    figsize=(11,6), color=[BLUE, ORANGE]
)

chart.invert_yaxis()
chart.set_xlim(0, 100)
chart.set_xlabel('Posts with this feature (%)')
chart.set_ylabel('')
chart.set_title('EDTT: visual features in videos and photo posts')
chart.legend(
    [
        f'Videos ({len(video_posts)} posts)',
        f'Photo posts ({len(photo_posts)} posts)'
    ],
    loc='upper center', bbox_to_anchor=(0.5, -0.12), ncol=2
)

finish('03_visual_format')
```



Performance and body language appear in 37 of 47 videos (78.7%) and 11 of 38 photo posts (28.9%). Media and memes show the reverse pattern: 10 videos (21.3%) and 27 photo posts (71.1%). Both differences are about 49.8 percentage points, the largest gaps in this sample.

Food imagery also appears more often in photo posts: 16 of 38 (42.1%), compared with 5 of 47 videos (10.6%). These differences describe the sampled posts. Changing format would not necessarily change a post's content.

5.4 Audio values

All 85 EDTT posts have these audio measurements. Tempo is musical speed in beats per minute. Valence estimates musical positivity. Arousal estimates energy. Both use a 0-to-1 scale and do not describe a person's feelings.

Summarize the values

The following tables summarize the average, spread, and quartiles of each audio measure. The median is the middle value. The 25th and 75th percentiles bound the middle half. We saved the complete table and displayed it in two parts for readability.

Code cell [23]

```
# We summarized audio values and give the table readable labels.
audio_columns = [
    'audio_tempo_bpm', 'audio_valence',
    'audio_arousal', 'harmonic_ratio'
]

audio_summary = tiktok[audio_columns].describe().T
audio_summary.index = [
    'Tempo (BPM)',
    'Musical positivity (valence)',
    'Musical energy (arousal)',
    'Tonal share (harmonic ratio)'

]

audio_summary = audio_summary.rename(columns={
    'count': 'Posts counted',
```

```
    'mean': 'Average',
    'std': 'Standard deviation',
    'min': 'Lowest',
    '25%': '25th percentile',
    '50%': 'Median',
    '75%': '75th percentile',
    'max': 'Highest'
})

display(audio_summary[['Posts counted', 'Average', 'Standard deviation', 'Median']].round(3))
display(audio_summary[['Lowest', '25th percentile', '75th percentile', 'Highest']].round(3))
audio_summary.to_csv(CLEAN / 'audio_summary.csv')
```

	Posts counted	Average	Standard deviation	Median
Tempo (BPM)	85.0	126.039	20.551	123.050
Musical positivity (valence)	85.0	0.590	0.261	0.736
Musical energy (arousal)	85.0	0.342	0.059	0.332
Tonal share (harmonic ratio)	85.0	0.742	0.185	0.789

	Lowest	25th percentile	75th percentile	Highest
Tempo (BPM)	78.300	112.350	143.550	172.270
Musical positivity (valence)	0.090	0.232	0.774	0.836
Musical energy (arousal)	0.205	0.303	0.385	0.497
Tonal share (harmonic ratio)	0.050	0.670	0.865	0.993

Plot the distributions

Each histogram uses ten equal-width bins across its observed values. The line marks the median. The boxplots show the middle half and possible extreme values. A point beyond a whisker is not automatically an error. We kept all rows.

Valence is the outcome for question 3. We examined it on its own, without separating posts by restraint status.

Code cell [24]

```
# Histograms show the shape; boxplots show the middle and spread.
plot_columns = [
    'audio_tempo_bpm', 'audio_valence', 'audio_arousal'
]
plot_labels = [
    'Tempo (BPM)',
    'Musical positivity (0–1)',
    'Musical energy (0–1)'
]

figure, charts = plt.subplots(
    2, 3, figsize=(12,6), layout='constrained'

)

for position, column in enumerate(plot_columns):
    values = tiktok[column]
    histogram = charts[0, position]
    boxplot = charts[1, position]

    values.hist(ax=histogram, bins=10, color=BLUE, edgecolor='white', grid=False)
    histogram.axvline(values.median(), color=ORANGE, label='Median')
    histogram.set_title(plot_labels[position])
```

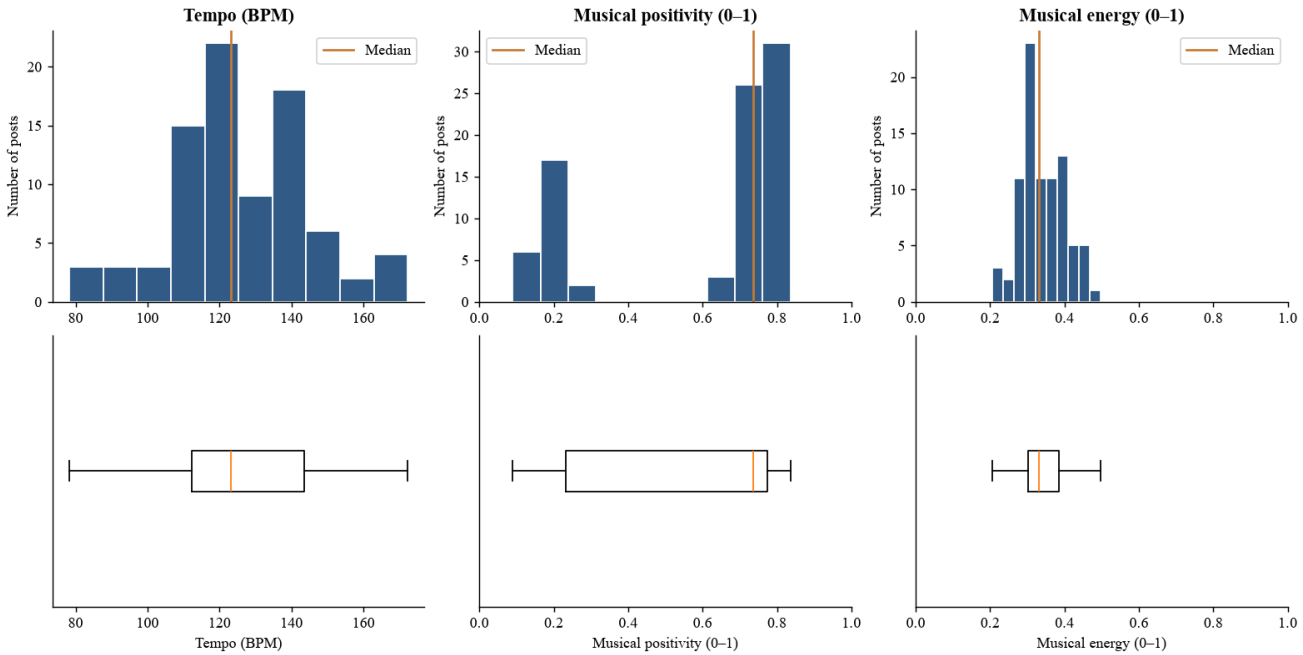
```
    histogram.set_ylabel('Number of posts')
    histogram.legend()

    boxplot.boxplot(values, orientation='horizontal')

    if column != 'audio_tempo_bpm':
        histogram.set_xlim(0, 1)

    boxplot.set_xlim(histogram.get_xlim())
    boxplot.set_yticks([])
    boxplot.set_xlabel(plot_labels[position])

finish('04_audio_distributions')
```



Median tempo is 123.05 BPM. The middle half lies between 112.35 and 143.55 BPM. The full range is 78.30 to 172.27 BPM.

Valence has a median of 0.736 and a lower mean of 0.590. The histogram shows both a higher-score group and a smaller lower-score group, so one average misses part of the distribution. The middle half spans 0.232 to 0.774.

Arousal is more concentrated, with its middle half between 0.303 and 0.385. Its spread should not be compared directly with tempo's because the units differ. These plots help us describe the planned outcome without testing the hypothesis.

5.5 What the EDA tells us

We retained the three research questions and the same rows. The figures answer different parts of question 1: label frequency, uncertainty from blank annotations, visual features by format, and audio distributions.

Shape concern is the main Twitter theme, while eating concern has a smaller but usable group. EDTT adds visual and audio measures, but only 79 posts have known EDE categories. These findings support one focused analysis per source rather than many comparisons.

Questions 2 and 3 remain untested. Separate label percentages do not show whether shape and eating concerns co-occur, and the audio distributions do not show whether restraint relates to valence. Neither dataset measures algorithm exposure or changes in food attitudes.

6. Questions for reviewers: Part 1

Our discussion raised questions about conflicting labels, missing annotations, and how the two datasets fit together.

1. Should we retain the 11 inconsistent Twitter rows and compare the later model with and without them, as planned?

2. Is leaving the six blank EDTT category fields unknown appropriate? Should we ask the source authors to clarify them before changing that rule?

3. Do the two separate datasets form a coherent project about food and body-image content, despite their different collection and coding methods?

4. Do the missing-label ranges and format comparisons help explain the data, or is another descriptive comparison more useful before modeling?

Part 2. Preregistration of future analyses

What has been viewed

These plans follow the data audit and EDA, not an untouched dataset. We inspected source documentation, examples, the Twitter consistency check, individual-variable summaries, and visual features by format. We have not tabulated, correlated, or modeled the two planned predictor-outcome pairs.

These plans were clarified on October 19 before either model was fitted. We will retain this dated version and document any later changes.

Reporting plan

We will report both analyses whether or not their results support the hypotheses. Each report will include the posts used, model coefficients, predicted values, and limitations. We will compare the direction and size of each relationship rather than treating a coefficient alone as proof of a hypothesis.

On October 20, before fitting either model, we simplified these plans to focus on regression coefficients and predictions. The hypotheses and variables remained unchanged. Formal significance tests are not part of this version of the plan. Any later addition will be documented before it is run.

Both models will use individual posts as observations. Sample size affects precision, but it does not remove selection bias, confounding, or dependence between posts.

7. Preregistration 1: Twitter concerns

Hypothesis

Among related Twitter posts, posts with a shape-concern label are more likely to have an eating-concern label. Body-image and food concerns may appear together, but this is a hypothesis about content rather than illness or causation.

Data and variables

We will use the 2,584 related Twitter posts. The outcome will be eating (0 or 1), and the predictor will be shape (0 or 1). We will retain the original labels and exclude any row missing either variable. The cleaning checks found no such missing values.

Planned analysis

We will first count posts with and without eating concern in each shape-concern group and calculate the corresponding percentages. We will then fit a logistic regression using LogisticRegression:

logit[P(eating = 1)] = beta_0 + beta_1 * shape

We will report the intercept and shape coefficient, then calculate the predicted probability of eating concern for shape = 0 and shape = 1. A positive coefficient and a higher predicted probability for the shape-concern group would agree with the hypothesis. A zero or negative difference would not support its expected direction.

We will use the model's default regularization settings and record the library version and settings in the final report. The fitted probabilities may therefore differ slightly from the observed group percentages. We will show both, without treating the model's predictions as a significance test.

Checks and reporting

We will include both group sizes and all four combinations of the labels, so small groups are visible. The main analysis will keep the 11 flagged records. We will repeat the same comparison without those records and report whether the result changes. If a variable has no variation or fitting fails, we will report that limitation instead of changing the hypothesis.

Limits

The relationship may reflect coding practices or repeated creators. Account type and post length were unavailable. We will not use noconcern as a control because it is defined from the concern labels themselves. The results will describe this sample and will not establish a causal effect or a relationship across all Twitter posts.

8. Preregistration 2: Restraint and audio

Hypothesis

EDTT posts with a recorded restraint label have lower average audio valence than posts without that label, after accounting for format. This asks whether food-restriction content has a different musical tone. Upbeat or ironic music could produce the opposite result.

Data and variables

We will use the 79 posts with nonblank EDE text and valid audio valence and format. The outcome will be audio_valence on its 0-to-1 scale. The predictors will be ede_restraint and a binary format variable, coded as 1 for photos and 0 for videos. The six posts with unknown EDE categories will remain excluded from the primary analysis.

Planned analysis

We will fit a multiple linear regression using LinearRegression:

audio_valence = beta_0 + beta_1 * ede_restraint + beta_2 * photos + error

We will report the intercept and both coefficients. Within each format, we will compare predicted valence for restraint = 0 and restraint = 1. A negative restraint coefficient would agree with the hypothesis because it represents lower predicted valence at the same format. A zero or positive coefficient would not support its expected direction.

We will also report the observed group sizes and mean valence values. These will provide context for the model's predictions, especially if some combinations of restraint and format contain few posts.

Checks and reporting

We will compare observed and predicted values, inspect the residuals, and calculate mean squared error to summarize how closely the model fits these posts. This will describe fit to the sample, not performance on new posts. We will report any predictions outside 0 to 1 and keep all eligible rows rather than removing posts to improve the result.

If either predictor has no variation or the two predictors cannot be distinguished, we will report that the planned comparison could not be estimated reliably. As a separate missing-data check, we will repeat the model with all six unknown restraint labels set to 0, then with all six set to 1. These will be hypothetical scenarios, not corrections to the original labels or guaranteed bounds on the effect.

Limits

Format will account for one observed difference between posts, but it will not control for every possible confounder. Account type, post length, and reliable creator identifiers were unavailable. The small sample and possible reuse of audio will limit interpretation. Musical valence will not be treated as a measure of viewers' emotions or health.

Appendix A. Data cleaning appendix and reproducibility

Files. data/raw/ contains unchanged source downloads and the checksum manifest. data/intermediate/ contains validated/normalized snapshots. data/processed/ contains analysis tables and audits. figures/ contains the four charts. The download bundle includes these files, the executed notebook, its HTML view, and the combined PDF. In GitHub, the notebook is in notebooks/ and the report is in reports/phase2/.

Re-run. Extract the full ZIP, install the packages in requirements.txt in a Python environment, open this notebook from the extracted directory, select that environment's kernel, and choose Restart Kernel and Run All. A complete local bundle runs offline. A notebook opened alone in Colab needs internet access on its first run to restore the saved inputs from GitHub. Processed files and figures are overwritten reproducibly. All code cells should finish without errors. No proposed inferential analyses are implemented here.

Hosted raw inputs. The verified GitHub raw-data folder contains both original datasets, the Twitter codebook and filtering appendix, and manifest.json. These are the exact bytes used in this execution. The publisher links in Section 2 provide independent access to the original releases.

Cleaning trace. The initial checks verified the four original files. Section 2.2 validated Twitter codes and their conditional universes, saves all 3,000 rows, retains 2,584 related posts, and flags 11 logical inconsistencies; Section 2.3 normalized all 85 EDTT rows and derives 11 category indicators plus a count while preserving six unknown EDE annotations; Section 2.4 saved row-flow and eligibility audits. Sections 5.1–5.4 generate the displayed summaries and figures from those analysis-ready files. Every code cell has an executed output or visible saved-file result, and the proposed inference remains unperformed.

Appendix B. References

1. Cavazos-Rehg, P. A., et al. (2019). “I just want to be skinny.”: A content analysis of tweets expressing eating disorder symptoms. *PLOS ONE*, 14(1), e0207506. https://doi.org/10.1371/journal.pone.0207506. S1 Dataset, S1 Codebook, and S1 Appendix are bundled unchanged.

2. Shaveet, E., Viranda, T., & Choudhury, T. (2026). *EDTT Dataset*, version 2. Zenodo. https://doi.org/10.5281/zenodo.19749827. Public dictionary: https://edenshaveet.github.io/EDTT/.

3. Project repository: https://github.com/AnyaChoudhary/info2950-fall2026-project. Raw-data snapshot linked in Section 2.

9. Questions for reviewers: Part 2

We would like feedback on whether the models match our questions and how to interpret their results within the limits of these samples.

1. Will the observed group percentages and logistic regression probabilities provide a clear comparison of eating concern across Twitter shape-concern groups?

2. Is multiple linear regression appropriate for EDTT valence after accounting for format, given its 0-to-1 scale and only 79 usable posts?

3. With account type, post length, and creator IDs unavailable, should we narrow the interpretation further or seek additional data before modeling?

4. Are the two assignments for the six unknown restraint labels useful checks, or should we first ask the source authors to clarify those blanks?

5. Are coefficients, predicted values, and fit summaries sufficient for the final report, or should we preregister an additional uncertainty analysis before fitting the models?

Both planned analyses will be reported even if their results do not support the hypotheses.